

App Configuration: Devicetree

Zephyr RTOS Course





Recap: L03 Kconfig



- `prj.conf` → `autoconf.h` → `CONFIG_*` macros in C
- `Kconfig` file in your app defines symbols
- `menuconfig` lets you browse and set options interactively
- Use `depends on`, hidden symbols, `choice`, `menu` to organize



Format



- Lecture with optional **live demo**
- Homework after each session (checked by us)
- Recordings will be shared
- Questions addressed at the end in a **Q&A session**



Where we left off



	Kconfig	Devicetree
Purpose	Software features	Hardware description
Question	"Do I want this feature?"	"Where is this hardware?"
Example	<code>CONFIG_LOG=y</code>	<code>zephyr,console = &uart0</code>
Files	<code>.conf</code> , <code>Kconfig</code>	<code>.dts</code> , <code>.overlay</code>
In code	<code>CONFIG_*</code> macros	<code>DT_*</code> macros



Devicetree



Devicetree



- Abstracts **board layout and peripherals**
- Describes what's on the board and how it's wired
- Independent of application logic



Devicetree: Syntax



Nodes, labels and properties



```
/ {  
    leds {  
        compatible = "gpio-leds"; // Tells Zephyr which driver to use  
  
        // node label: node name  
        // label - shortcut for referencing  
        // name - identifies node in tree  
        blue_led: led_name {  
            // &hsgpio0 - reference to another node  
            gpios = <&hsgpio0 0 GPIO_ACTIVE_LOW>;  
            label = "Blue LED"; // Key-value data  
        };  
    };  
};
```



Property value types



```
node {  
    // String  
    label = "Blue LED";  
    compatible = "gpio-leds";  
  
    // Integer (in angle brackets)  
    clock-frequency = <48000000>;  
  
    // Array of integers  
    reg = <0x40000000 0x1000>;  
  
    // Phandle + args (reference to another node)  
    gpios = <&hsgpio0 0 GPIO_ACTIVE_LOW>;  
  
    // Boolean (present = true, absent = false)  
    read-only;  
};
```



Devicetree: Overlays



What are overlays?



Patch files that modify the base board Devicetree — add new nodes or override existing properties.

```
/* boards/frdm_rw612.overlay */
/ {
    leds {
        blue_led: led_name {
            gpios = <&hsgpio0 0 GPIO_ACTIVE_LOW>;
        };
    };
};
```



Overlay loading order



Auto-discovery (when `DTC_OVERLAY_FILE` is not set):

1. `boards/<BOARD>.overlay` — matched by board name
2. `app.overlay` — fallback if no board overlay found

Stops at first match — if `boards/` overlay exists, `app.overlay` is ignored.



Override auto-discovery



Force a specific overlay:

```
west build -- -DDTC_OVERLAY_FILE="boards/frdm_rw612_red.overlay"
```

Add extra overlay on top of auto-discovered ones:

```
west build -- -DEXTRA_DTC_OVERLAY_FILE="my_extra.overlay"
```



Inspecting the Generated DTS



```
build/zephyr/zephyr.dts
```

- Final merged devicetree (board DTS + all overlays applied)
- Verify your overlay was correctly merged
- Find the actual node path for `DT_PATH()`
- Check that aliases point to the right nodes



Devicetree: Aliases



What are aliases?



Named shortcuts to nodes — a stable public API for your app

```
/ {  
    aliases {  
        led0 = &blue_led;      // by node label  
        led1 = "/leds/led_red"; // by path  
    };  
};
```



Accessing nodes from C code



Three ways to get a node reference:

```
// By alias (preferred – stable API)
#define LED_NODE DT_ALIAS(led0)

// By node label (when no alias exists)
#define LED_NODE DT_NODELABEL(blue_led)

// By path (rarely used)
#define LED_NODE DT_PATH(leds, led_name)
```



Demo: Add Missing LEDs



The board DTS only defines a green LED. Add blue and red LEDs via overlay.

1. Create `boards/frdm_rw612.overlay` with blue and red LED nodes
2. Override `led0` alias to point to blue LED
3. Access the LED using `DT_ALIAS(led0)` – verify it blinks

Tip: Check `dts/bindings/gpio/gpio-leds.yaml` to see which properties are available

Tip: Check pin assignment in the DK schematic



Homework: Kconfig + Overlay Integration



Homework: Heartbeat LED via Kconfig + Overlay



Add a configurable heartbeat LED to the blinky app from the demo. Works on any board.

- Create `app.overlay` — add alias `app-led` pointing to your board's `led0`
- Add `Kconfig` file with;
 - `int APP_HEARTBEAT_PERIOD_MS` (default `500`, range `100` — `2000`)
- In C: use `DT_ALIAS(app_led)` for the GPIO and `CONFIG_APP_HEARTBEAT_PERIOD_MS` for the sleep duration
- Verify: open `menuconfig`, change the period, rebuild — LED blink speed must change
- Push tag: `l4-task1`



Best Practices



Devicetree



- ✓ Use **aliases** for your app's public API (`led0`, `warning_led`)
- ✓ Keep overlays in `boards/` – committed to git
- ✓ Label nodes you'll reference from code
- ✓ Use `compatible` to match the right driver
- ✗ Don't hardcode GPIO pins in C code
- ✗ Don't commit generated DTS (`build/zephyr/zephyr.dts`)

Thank You!

Questions?





Resources

1. [Devicetree Guide](#)
2. [Devicetree Overlays How-To](#)
3. [Kconfig Reference](#)

